CHƯƠNG 6

Kiến trúc ứng dụng AI tạo sinh

[Ứng dụng Trí tuệ nhân tạo]



[Nguyễn Tuấn Anh]
- [Thái Nguyên, 2026]

Giới thiệu nội dung

Mục tiêu chương

- Mô tả được kiến trúc tổng thể của ứng dụng AI tạo sinh theo mô hình phân tầng 5 tầng và vai trò của từng tầng.
- Phân tích được vai trò và đặc điểm của bốn thành phần chính: mô hình (model), API, dữ liệu (data) và pipeline.
- So sánh được ba mô hình kiến trúc phổ biến (API-based,

Nội dung chính

1. Kiến trúc tổng thể của ứng dụng AI tạo sinh
2. Các thành phần hệ thống AI tạo sinh
3. Các mô hình kiến trúc phổ biến
4. Thiết kế kiến trúc và phân tích trade-off

Khởi động: Điều gì làm nên một ứng dụng AI tạo sinh?

Ứng dụng web truyền thống

- Hành vi xác định bởi **mã nguồn**.
- Mỗi đầu vào $\rightarrow$ một đầu ra **có thể dự đoán**.
- Logic nghiệp vụ được lập trình **tường minh**.

Ứng dụng AI tạo sinh (như ChatGPT)

- Một phần "logic" nằm trong **LLM** – thực thể **xác suất**.
- Cùng một input có thể tạo ra các output **khác nhau**.
- Hành vi được điều khiển qua prompt, không qua code.

Câu hỏi kiến trúc mới

Làm thế nào để quản lý tính không xác định? Kết hợp sức mạnh LLM với ràng buộc nghiệp vụ? Đảm bảo chất lượng khi mô hình có thể "bịa ra" thông tin?

Ba khác biệt căn bản với ứng dụng truyền thống

1. **Tính xác định vs Tính xác suất**
   - Truyền thống: cùng input $\rightarrow$ cùng output – kiểm thử chính xác, gỡ lỗi theo vết.
   - AI tạo sinh: LLM là hệ thống xác suất – ngay cả với `temperature = 0` vẫn có biến thiên. Thách thức cho kiểm thử và giám sát.
2. **Nguồn gốc của "logic" hệ thống**
   - Truyền thống: logic nằm trong mã nguồn – đọc được, sửa được, version control.
   - AI tạo sinh: logic nằm trong **trọng số mô hình** + **prompt** – không thể "đọc" hay "sửa" trực tiếp. Điều khiển qua prompt engineering, cấu hình tham số, thiết kế pipeline.
3. **Chi phí vận hành và độ trễ**
   - Truy vấn CSDL: mili-giây. Lời gọi LLM: **hàng trăm ms đến hàng chục giây**.
   - Chi phí mỗi lời gọi LLM cao hơn **nhiều bậc** so với truy vấn CSDL.

Kiến trúc phân tầng của ứng dụng AI tạo sinh

- Hầu hết ứng dụng AI tạo sinh tuân theo **kiến trúc phân tầng** (layered architecture) với 5 tầng chức năng chính.

Tầng Trải nghiệm (Experience Layer) Giao diện người dùng (Web, Mobile, Chat), voice, API endpoint
Tầng Điều phối (Orchestration Layer) Quản lý hội thoại, prompt pipeline, chain/tool execution, state
Tầng AI / Mô hình (AI / Model Layer) LLM serving, model routing, embedding, fine-tuning, guardrails
Tầng Dữ liệu & Tri thức (Data & Knowledge Layer) Vector database, knowledge graph, document store, cache
Tầng Hạ tầng (Infrastructure Layer) Compute (GPU/TPU), networking, monitoring, logging, security

Chi tiết tầng Trải nghiệm & tầng Điều phối

Tầng Trải nghiệm

- **Hiển thị nội dung sinh**: Markdown, code block, bảng biểu, hình ảnh – tất cả được sinh động.
- **Tương tác hội thoại**: duy trì đối thoại nhiều lượt, streaming, cho phép dừng/chỉnh sửa.
- **Cơ chế phản hồi**: thumbs up/down, báo cáo lỗi, yêu cầu sinh lại.

Tầng Điều phối – "Bộ não điều hành"

- **Quản lý hội thoại**: sliding window, rolling summary.
- **Thực thi pipeline**: retrieval $\rightarrow$ prompt $\rightarrow$ LLM call $\rightarrow$ post-processing.
- **Định tuyến mô hình**: mô hình nhẹ cho đơn giản, mô hình mạnh cho phức tạp.
- **Kiểm soát chất lượng**: guardrails, output validation, fallback.

Framework: LangChain, LlamaIndex, Semantic Kernel, hoặc custom orchestration.

Chi tiết tầng AI/Mô hình & tầng Dữ liệu

Tầng AI / Mô hình

- **Model Serving**: cân bằng tải, auto-scaling, quản lý phiên bản.
- **Model Gateway**: giao diện thống nhất cho nhiều provider (OpenAI, Anthropic, Google).
- **Embedding Service**: chuyển văn bản thành vector.
- **Guardrails**: lọc nội dung độc hại, phát hiện PII, chống prompt injection.

Tầng Dữ liệu & Tri thức

- **Vector Database**: Pinecone, Weaviate, Milvus, Qdrant, pgvector, Chroma.
- **Document Store**: S3, GCS cho tài liệu gốc (PDF, Markdown).
- **Knowledge Graph**: biểu diễn quan hệ thực thể – hữu ích cho suy luận phức tạp.
- **Context Store / Cache**: Redis – giảm độ trễ và chi phí qua semantic caching.

Quyết định then chốt

API bên thứ ba (OpenAI, An-

Nguyên lý thiết kế cốt lõi cho kiến trúc AI tạo sinh

1. **Thiết kế cho tính không xác định**
   - Fault-tolerant parsing, cơ chế retry với prompt điều chỉnh, structured output (JSON mode, function calling), fallback path.
2. **Tối ưu cho độ trễ và chi phí**
   - Semantic caching, token streaming, mô hình phân tầng (nhẹ cho đơn giản, mạnh cho phức tạp), batch processing, xử lý bất đồng bộ.
3. **Kiểm soát và giám sát toàn diện**
   - Observability (độ trễ + chất lượng AI), guardrails đa tầng, audit trail, human-in-the-loop.
4. **Module hóa và khả năng thay thế**
   - Cô lập phụ thuộc mô hình sau interface trừu tượng, model swapping, A/B testing, cập nhật prompt không cần redeploy.

Ví dụ minh họa: Kiến trúc ứng dụng hỏi đáp tài liệu (RAG)

1. **Tầng Trải nghiệm**: Người dùng đặt câu hỏi qua giao diện chat web. Gửi qua WebSocket để hỗ trợ streaming.
2. **Tầng Điều phối**:
    1. Chuyển câu hỏi thành embedding query.
    2. Truy vấn vector database – top-k retrieval.
    3. Xây dựng prompt: system prompt + context + history + question.
    4. Gọi LLM qua model gateway $\rightarrow$ stream kết quả.
    5. Lưu tương tác vào context store.
3. **Tầng AI/Mô hình**: Embedding service + LLM service. Model gateway định tuyến theo loại yêu cầu.
4. **Tầng Dữ liệu**: Vector DB (Pinecone/pgvector) + Document store (S3) + Context store (Redis).
5. **Tầng Hạ tầng**: Kubernetes cluster với GPU nodes, giám sát bởi Datadog và LangSmith.

Tổng kết Section 1

Những điểm cần nhớ

1. **Ba khác biệt căn bản**: Tính không xác định, nguồn gốc logic (trọng số + prompt), chi phí/độ trễ cao – chi phối mọi quyết định kiến trúc.
2. **Kiến trúc 5 tầng**: Trải nghiệm – Điều phối – AI/Mô hình – Dữ liệu & Tri thức – Hạ tầng. Mỗi tầng có vai trò và công nghệ riêng.
3. **Bốn nguyên lý thiết kế**: (1) Thiết kế cho tính không xác định; (2) Tối ưu độ trễ & chi phí; (3) Giám sát toàn diện; (4) Module hóa để dễ thay thế.

Tự kiểm tra nhanh:

1. Kể tên 5 tầng trong kiến trúc phân tầng. Tầng nào là “bộ não điều hành”?
2. Tại sao kiến trúc AI cần được thiết kế cho “tính không xác định”?

Khởi động: “Con quái vật nhiều đầu”

- Sai lầm phổ biến: chỉ tập trung vào LLM – “bộ não” – mà bỏ qua các thành phần khác.
- LLM chỉ là **một thành phần** trong hệ sinh thái phức tạp. Nếu không có các thành phần hỗ trợ phù hợp:
  - Phản hồi chậm, thông tin lỗi thời, thiếu ngữ cảnh, thậm chí gây hại.

Bốn thành phần chính của hệ thống AI tạo sinh:

Mô hình (Model)	**API** (API)	**Dữ liệu** (Data)	**Pipeline** (Pipeline)
“Động cơ”	“Cầu nối”	“Nhiên liệu”	“Dây chuyền”

Mô hình (Model) – "Trái tim" của hệ thống

Phân loại mô hình theo vai trò:

- **Mô hình chính** (Primary Model): LLM mạnh nhất – GPT-4o, Claude Opus, Gemini Pro. Tạo đầu ra cuối cùng.
- **Mô hình nhẹ** (Lightweight Model): GPT-4o mini, Claude Haiku, Gemini Flash. Phân loại ý định, tóm tắt ngắn, tiền xử lý.
- **Mô hình chuyên biệt** (Specialized Model): Embedding model (text-embedding-3-small), reranking (Cohere Rerank), moderation.
- **Mô hình đa phương thức** (Multimodal Model): GPT-4o (vision + text), Gemini Pro Vision – xử lý nhiều loại dữ liệu cùng lúc.

Chiến lược mô hình phân tầng

Bài toán tối ưu đa mục tiêu: **chất lượng**, **tốc độ** và **chi phí**. Dùng mô hình nhẹ cho tác vụ đơn giản, mô hình mạnh cho suy luận phức tạp.

Model-as-a-Service vs Self-Hosting

Tiêu chí	Model-as-a-Service (API)	Self-Hosting
Chi phí khởi tạo	Thấp – trả theo usage, không cần GPU	Cao – cần GPU, hạ tầng, nhân sự
Chi phí vận hành	Tuyến tính theo usage	Cố định + biến đổi; hiệu quả ở quy mô lớn
Độ trễ	Phụ thuộc nhà cung cấp, có thể biến động	Kiểm soát được, tối ưu cho use case cụ thể
Kiểm soát	Hạn chế – phụ thuộc model version	Toàn quyền – fine-tune, quantization, routing
Bảo mật dữ liệu	Dữ liệu rời khỏi tổ chức	Dữ liệu nằm trong hạ tầng của tổ chức
Cập nhật mô hình	Tự động – luôn dùng model mới nhất	Thủ công – cần quy trình kiểm thử

Chiến lược Hybrid: API cho tác vụ thông thường + Self-host cho tác vụ nhạy cảm về dữ liệu hoặc yêu cầu đặc thù về độ trễ.

Model Gateway – Lớp trừu tượng hóa mô hình

- Khi dùng nhiều mô hình từ nhiều provider, tích hợp trực tiếp $\rightarrow$ mã nguồn phức tạp, khó bảo trì.
- **Model Gateway** cung cấp **giao diện thống nhất** (unified interface) cho tất cả mô hình.

Trách nhiệm chính:

1. **Chuẩn hóa giao diện**: chuyển đổi giữa định dạng nội bộ và API của từng provider.
2. **Định tuyến thông minh**: chọn mô hình dựa trên loại tác vụ, độ phức tạp, ngân sách token.
3. **Fallback & Retry**: tự động thử mô hình khác nếu mô hình chính lỗi hoặc quá tải.
4. **Giám sát tập trung**: metrics về usage, độ trễ, chi phí, chất lượng từ một điểm duy nhất.
5. **Quản lý API key**: tập trung hóa việc quản lý và luân chuyển key.

Ví dụ

API – Các loại API trong hệ thống AI tạo sinh

- **Inference API**: API cốt lõi để gọi mô hình – gửi prompt và nhận phản hồi. Hỗ trợ cả chế độ đồng bộ và streaming (SSE). OpenAI Chat Completion API là **tiêu chuẩn de facto**.
- **Embedding API**: chuyển đổi văn bản thành vector embedding. Dùng nội bộ trong pipeline RAG, không tiếp xúc trực tiếp với người dùng cuối.
- **Management API**: quản lý tài nguyên AI – fine-tuning jobs, model deployments, file uploads (cho RAG), assistants/agents configuration.
- **Application API**: API hướng đến người dùng cuối hoặc frontend – `/chat`, `/search`, `/summarize`. Đóng gói toàn bộ logic nghiệp vụ.

MCP (Model Context Protocol) và A2A (Agent-to-Agent) là hai giao thức đang định hình cách các thành phần AI giao tiếp – giúp tránh vendor lock-in.

API – Những điểm đặc thù cần lưu ý

- **Streaming response**: thay vì JSON hoàn chỉnh, API cần hỗ trợ SSE hoặc WebSocket để stream từng token – cải thiện đáng kể UX.
- **Long-running requests**: tác vụ AI có thể mất hàng chục giây đến vài phút. API nên hỗ trợ mô hình bất đồng bộ: task ID + endpoint kiểm tra trạng thái.
- **Rate limiting theo token**: do chi phí cao của mỗi lời gọi LLM, cần rate limiting theo cả số request và số token sử dụng.
- **Error handling đặc thù**: content filter triggered, context length exceeded, model overloaded – mỗi loại lỗi cần chiến lược xử lý riêng.
- **Versioning**: prompt hoặc model thay đổi $\rightarrow$ hành vi API thay đổi đáng kể. Cần versioning rõ ràng và A/B testing.

Dữ liệu (Data) – "Nhiên liệu" của hệ thống AI

Các loại dữ liệu trong ứng dụng AI tạo sinh:

- **User Data**: hội thoại, tùy chọn, phản hồi – PostgreSQL, MongoDB.
- **Knowledge Base**: tài liệu, FAQ – S3 + Vector DB (cốt lõi của RAG).
- **Embedding Data**: vector biểu diễn văn bản – cầu nối ngôn ngữ tự nhiên $\leftrightarrow$ toán học.
- **Observability Data**: log, trace, metrics để gỡ lỗi và tối ưu.
- **Training & Eval Data**: tập dữ liệu fine-tune và đánh giá chất lượng.

Chất lượng và cách tổ chức dữ liệu quyết định phần lớn chất lượng đầu ra, đặc biệt trong các hệ thống RAG ("garbage in, garbage out").

Chiến lược Indexing và Chunking

Hiệu quả của RAG phụ thuộc rất nhiều vào cách tài liệu được chia nhỏ (chunking) và lập chỉ mục (indexing):

- **Fixed-size chunking**: chia tài liệu thành các đoạn có kích thước cố định (512 token) với overlap (50 token). Đơn giản, dễ triển khai nhưng có thể cắt ngang giữa các ý.
- **Semantic chunking**: chia dựa trên ranh giới ngữ nghĩa (hết đoạn văn, hết mục). Chất lượng cao hơn nhưng phức tạp hơn.
- **Hierarchical chunking**: tạo nhiều mức chunk (đoạn nhỏ, mục, chương), cho phép truy xuất ở nhiều mức độ chi tiết.
- **Agentic chunking**: sử dụng LLM để xác định điểm cắt tối ưu. Chất lượng cao nhất nhưng chi phí indexing cũng cao nhất.
- **Multi-representation indexing**: lưu trữ nhiều biểu diễn của cùng một tài liệu (tóm tắt, đầy đủ, trích đoạn chính).

Không có chiến lược "tốt nhất" cho mọi trường hợp – cần **thử nghiệm và điều chỉnh liên tục**.

Vector Database – Thành phần cốt lõi của RAG

Nguyên lý hoạt động:

1. Văn bản $\rightarrow$ embedding model $\rightarrow$ vector.
2. Lưu vector + metadata vào vector database.
3. Khi có truy vấn: câu hỏi $\rightarrow$ embedding.
4. Tìm kiếm nearest neighbor (cosine similarity / Euclidean distance).
5. Trả về tài liệu tương ứng làm ngữ cảnh cho LLM.

Database	**Loại**	**Phù hợp cho**
Pinecone	Managed cloud	Startup, PoC nhanh
Weaviate	Managed / Self-host	Hybrid search (vector + keyword)
Milvus	Self-host (có managed)	Quy mô lớn, enterprise
Qdrant	Self-host / Managed	Performance & flexibility
pgvector	PostgreSQL extension	Ứng dụng đã dùng PostgreSQL
Chroma	Embedded / Self-host	Prototyping, development

Pipeline – “Dây chuyền” xử lý thông minh

Bốn mô hình pipeline cơ bản:

1. **Tuyến tính** (Linear): Input $\rightarrow$ Preprocessing $\rightarrow$ Embedding $\rightarrow$ Search $\rightarrow$ Prompt $\rightarrow$ LLM $\rightarrow$ Output. Phổ biến nhất, phù hợp hầu hết use case cơ bản.
2. **Phân nhánh** (Branching): rẽ nhánh dựa trên kết quả trung gian. VD: phân loại ý định $\rightarrow$ RAG / LLM trực tiếp / tool-use.
3. **Vòng lặp** (Loop / Agentic): LLM sinh $\rightarrow$ đánh giá chất lượng $\rightarrow$ nếu chưa đạt, sinh lại với prompt điều chỉnh. Hiện thực của mẫu Reflection (Chương 5).
4. **Song song** (Parallel): nhiều nhánh chạy đồng thời, tổng hợp kết quả. VD: truy vấn đồng thời Vector DB + Knowledge Graph + SQL.

Ví dụ: Pipeline RAG chi tiết (12 bước)

1. **Input Guard**: kiểm tra nội dung độc hại, PII, prompt injection.
2. **Intent Classification** (Lightweight Model): phân loại ý định.
3. **Query Rewriting** (LLM): viết lại câu hỏi để tối ưu truy xuất.
4. **Embedding**: chuyển câu hỏi thành vector.
5. **Vector Search**: top-k đoạn tài liệu liên quan nhất.
6. **Reranking** (Specialized Model): sắp xếp lại theo mức độ liên quan thực sự.
7. **Context Filtering**: lọc bỏ đoạn dưới ngưỡng.
8. **Prompt Assembly**: system prompt + context + history + question.
9. **LLM Inference** (Primary Model): gọi LLM, stream kết quả.
10. **Output Validation**: kiểm tra chất lượng, phát hiện hallucination.
11. **Output Guard**: kiểm tra an toàn đầu ra.
12. **Response + Citations**: trả về kèm trích dẫn nguồn.

Mỗi bước có thể bật/tắt hoặc cấu hình độc lập – linh hoạt theo đặc thù ứng dụng.

Tổng kết Section 2

Những điểm cần nhớ

1. **Bốn thành phần**: Mô hình (động cơ), API (cầu nối), Dữ liệu (nhiên liệu), Pipeline (dây chuyền) – phải hoạt động đồng bộ.
2. **Model Gateway** là lớp trừu tượng then chốt: chuẩn hóa giao diện, định tuyến thông minh, fallback, giám sát tập trung.
3. **Vector Database** là công nghệ cốt lõi của RAG – tìm kiếm ngữ nghĩa thay vì khớp từ khóa.
4. **Pipeline** kết nối mọi thành phần: tuyến tính, phân nhánh, vòng lặp, song song. Mỗi bước có thể cấu hình độc lập.

Tự kiểm tra nhanh:

1. Phân biệt 4 loại mô hình theo vai trò trong ứng dụng AI.
2. Model Gateway giải quyết vấn đề gì? Kể 3 trách nhiệm.
3. So sánh pipeline tuyến tính và pipeline vòng lặp (agentic).

Khởi động: Không có "viên đạn bạc" trong kiến trúc AI

- "Nên dùng kiến trúc gì?" – **Tùy vào hoàn cảnh**.
- Startup xây chatbot đơn giản $\neq$ Ngân hàng tích hợp AI vào core banking $\neq$ Công ty SaaS xây nền tảng AI đa tenant.

Ba mô hình kiến trúc phổ biến:

1. **API-based**: Đơn giản nhất – gọi trực tiếp LLM API. Điểm khởi đầu cho hầu hết ứng dụng.
2. **Microservices**: Phân tách hệ thống thành các dịch vụ độc lập. Cho ứng dụng phức tạp, quy mô lớn.
3. **RAG-based**: LLM + hệ thống truy xuất thông tin. Cho ứng dụng hỏi đáp tài liệu.

Ba mô hình **không loại trừ lẫn nhau** – có thể kết hợp và tiến hóa theo thời gian.

Mô hình 1: Kiến trúc API-based

Mô tả: Backend đóng vai trò **proxy** – nhận yêu cầu, thêm system prompt, gọi LLM API, trả kết quả.

Ưu điểm

- Triển khai nhanh – xây dựng MVP trong vài ngày.
- Chi phí khởi tạo thấp – không cần GPU.
- Luôn cập nhật mô hình mới nhất từ provider.
- Đơn giản về vận hành – không quản lý GPU cluster.

Nhược điểm

- Hạn chế kiểm soát – prompt engineering là công cụ chính.
- Vendor lock-in – phụ thuộc vào API của một provider.
- Chi phí tăng nhanh ở quy mô lớn.
- Rủi ro về quyền riêng tư dữ liệu.

Mô hình 2: Kiến trúc Microservices cho AI

Mô tả: Phân tách hệ thống thành các dịch vụ độc lập, mỗi dịch vụ đảm nhận một chức năng cụ thể:

- **Chat Service**: quản lý hội thoại, streaming. **RAG Service**: embedding, vector search, reranking.
- **Model Gateway Service**: model routing, provider abstraction. **Document Service**: upload, parsing, chunking, indexing.
- Mỗi service có database riêng + Message Queue (Kafka/RabbitMQ) cho giao tiếp bất đồng bộ.

Ưu điểm

- Scale độc lập từng service.
- Công nghệ phù hợp cho từng service.
- Độc lập triển khai, cô lập

Nhược điểm

- Vận hành phức tạp hơn nhiều (Kubernetes).
- Độ trễ liên service, data consistency phức tạp.
- Over-engineering cho ứng

Mô hình 3: Kiến trúc RAG-based

Mô tả: LLM được "tăng cường" bởi hệ thống truy xuất thông tin, cung cấp ngữ cảnh liên quan cho mỗi truy vấn. **Bốn thành phần cốt lõi**:

1. **Ingestion Pipeline**: thu thập tài liệu $\rightarrow$ parsing $\rightarrow$ chunking $\rightarrow$ embedding $\rightarrow$ indexing (chạy bất đồng bộ).
2. **Retrieval Engine**: query rewriting, embedding, vector search, reranking, hybrid search.
3. **Generation Engine**: xây dựng prompt từ kết quả truy xuất, gọi LLM, trả về kèm trích dẫn nguồn.
4. **Feedback Loop**: thu thập phản hồi, phân tích truy vấn kém, cập nhật chiến lược.

Tổng kết

$\oplus$ Giảm hallucination, tri thức luôn cập nhật, minh bạch (trích dẫn nguồn), bảo mật dữ liệu. $\ominus$ Pipeline phức tạp, chất lượng phụ thuộc dữ liệu đầu vào, chi phí lưu trữ.

Các biến thể của kiến trúc RAG

1. **Naive RAG**: chunk → embed → search → generate. Cơ bản nhất, phù hợp PoC.
2. **Advanced RAG**: thêm query rewriting, multi-hop retrieval, recursive retrieval. Độ chính xác cao hơn.
3. **Modular RAG**: các thành phần được module hóa, hoán đổi độc lập – dễ thử nghiệm và tối ưu.
4. **Agentic RAG**: tích hợp với AI Agent – agent tự quyết định khi nào truy xuất, từ đâu, đã đủ chưa.
5. **Graph RAG**: kết hợp Knowledge Graph – trả lời câu hỏi suy luận về mối quan hệ thực thể.

Xu hướng

Agentic RAG và Graph RAG là hướng phát triển tiên tiến nhất, cho phép suy luận phức tạp và tự chủ trong việc truy xuất thông tin.

Case Study: Ba tình huống thực tế

Tiêu chí	Tình huống 1: AI Writing Assistant	Tình huống 2: Ngân hàng – Trợ lý nội bộ	Tình h tảng A
Yêu cầu chính	Sinh văn bản, nhiều tông giọng	Hỏi đáp tài liệu nội bộ, bảo mật tuyệt đối	Đa ten model
Ràng buộc	Startup, nguồn lực hạn chế, ra mắt nhanh	Dữ liệu không rời hạ tầng, độ chính xác cao	Scale iting
Lựa chọn	**API-based**	**RAG + Self-hosting**	**Micros way**
Lý do	Đơn giản, nhanh, tập trung vào UX và prompt	Bảo mật là sống còn, RAG cho trích dẫn nguồn	Cần i scale

Nguyên tắc: Bắt đầu đơn giản, phát triển kiến trúc cùng với sự trưởng thành của sản phẩm và đội ngũ.

So sánh ba mô hình kiến trúc

Tiêu chí	API-based	Microservices	RAG-based
Độ phức tạp	Thấp	Cao	Trung bình – Ca
Thời gian ra MVP	1–2 tuần	2–4 tháng	3–8 tuần
Chi phí khởi tạo	Thấp	Cao	Trung bình
Khả năng mở rộng	Hạn chế	Rất tốt	Tốt
Kiểm soát mô hình	Hạn chế (prompt)	Linh hoạt	Linh hoạt
Giảm hallucina-tion	Không có cơ chế	Tùy thiết kế	Có (neo vào ngu
Trích dẫn nguồn	Không	Có thể	Có (cốt lõi)
Phù hợp nhất	MVP, startup, app đơn giản	Hệ thống phức tạp, đa team, đa tenant	Hỏi đáp tài liệu, thức

Tổng kết Section 3

Những điểm cần nhớ

1. **API-based**: Điểm khởi đầu cho mọi ứng dụng – đơn giản, nhanh, chi phí thấp. Hạn chế: vendor lock-in, ít kiểm soát.
2. **Microservices**: Cho hệ thống phức tạp, quy mô lớn – scale độc lập, phân công nhóm rõ ràng. Đánh đổi: vận hành phức tạp.
3. **RAG-based**: Giảm hallucination, trích dẫn nguồn, tri thức luôn cập nhật. Đánh đổi: pipeline phức tạp, phụ thuộc chất lượng dữ liệu.
4. **Không loại trừ lẫn nhau**: Có thể bắt đầu API-based $\rightarrow$ thêm RAG $\rightarrow$ chuyển dần sang Microservices.

Tự kiểm tra nhanh:

1. Khi nào nên chọn API-based thay vì Microservices?
2. RAG-based giải quyết vấn đề gì mà hai mô hình kia không làm

Khởi động: Nghệ thuật của sự đánh đổi

- *"Bạn sẽ thiết kế chatbot cho 10 triệu người dùng như thế nào?"*
- *"Còn nếu bạn chỉ có 2 tuần và 1 lập trình viên?"*

Sự thật quan trọng

Không có kiến trúc nào là "tốt nhất" một cách tuyệt đối. Thiết kế kiến trúc giỏi không phải là chọn giải pháp "tốt nhất về mặt kỹ thuật", mà là chọn giải pháp **tối ưu cho ràng buộc cụ thể**: thời gian, ngân sách, nhân sự, yêu cầu chất lượng và bối cảnh kinh doanh.

Bốn nguyên tắc thiết kế kiến trúc

1. **Bắt đầu từ bài toán, không phải từ công nghệ**
   - Trước khi nghĩ đến công nghệ: Người dùng là ai? Yêu cầu chất lượng? Ràng buộc? Quy mô? Điều gì sẽ thay đổi?
2. **Thiết kế cho sự thay đổi**
   - AI thay đổi hàng tuần: abstraction layer cho model provider, prompt template bên ngoài code, module hóa để mở rộng, observability ngay từ đầu.
3. **Đơn giản hóa cho đến khi không thể đơn giản hơn**
   - Mỗi thành phần thêm vào = chi phí (viết, test, deploy, monitor, debug, bảo trì). YAGNI: You Ain't Gonna Need It.
4. **Ưu tiên trải nghiệm người dùng**
   - Streaming **không phải optional**. TTFT (Time To First Token) < 500ms. Xử lý lỗi thân thiện. Cơ chế undo/retry.

Trade-off 1: Chất lượng – Độ trễ

- **Chọn mô hình**: GPT-4o/Claude Opus (chất lượng cao, chậm) vs GPT-4o mini/Claude Haiku (nhanh, có thể thiếu chính xác).
- **Số bước pipeline**: mỗi bước thêm (rewriting, reranking, validation) cải thiện chất lượng nhưng tăng độ trễ. Pipeline 12 bước $\rightarrow$ 5–10 giây.
- **Retrieval depth**: top-20 thay vì top-5 – tăng khả năng tìm thấy nhưng tăng token và độ trễ.
- **Streaming vs Batch**: streaming cải thiện UX nhưng tổng thời gian có thể lâu hơn, phức tạp hơn để monitor.

Chiến lược cân bằng

Mô hình phân tầng + “early exit” trong pipeline + streaming với chunking thông minh.

Trade-off 2: Chi phí – Năng lực

Mô hình	Input ($/1M token)	Output ($/1M token)	Ghi chú
GPT-4o mini	0,15	0,60	Nhẹ, nhanh, rẻ
GPT-4o	2,50	10,00	Đa năng cân bằng
Claude Haiku	0,25	1,25	Nhẹ của Anthropic
Claude Sonnet	3,00	15,00	Cân bằng
Claude Opus	15,00	75,00	Mạnh nhất, suy luận sâu
Gemini Flash	0,15	0,60	Nhẹ của Google
Gemini Pro	1,25	5,00	Cân bằng của Google

Lưu ý

Mô hình mạnh có thể đắt gấp **10–50 lần** mô hình nhẹ. Cần chiến lược rõ ràng để tối ưu chi phí.

Chiến lược

Model cascading (bắt đầu từ rẻ nhất) + Semantic caching (tiết kiệm 30–50%) + Batch processing (discount 50%) + Self-hosting tại điểm hòa vốn.

Trade-off 3: Đơn giản – Linh hoạt

- **Monolith vs Microservices**: monolith đơn giản để phát triển; microservices linh hoạt nhưng phức tạp để vận hành.
- **Framework vs Custom**: LangChain giúp phát triển nhanh nhưng khó tùy chỉnh sâu; custom implementation linh hoạt tuyệt đối nhưng tốn thời gian.
- **Managed vs Self-hosted**: managed service đơn giản trong vận hành; self-hosted linh hoạt hoàn toàn nhưng đòi hỏi năng lực vận hành.
- **Cứng nhắc vs Động**: pipeline cố định dễ hiểu, dễ gỡ lỗi; agentic pipeline linh hoạt nhưng khó kiểm soát và dự đoán.

Chiến lược cân bằng

Bắt đầu với lựa chọn đơn giản hơn. Chỉ tăng độ phức tạp khi có bằng chứng rõ ràng rằng sự đơn giản hiện tại đang cản trở mục tiêu.

Trade-off 4: Kiểm soát – Tiện lợi

Đặc biệt quan trọng khi lựa chọn giữa dịch vụ bên ngoài và tự xây dựng:

- **API bên thứ ba**: tiện lợi – không cần quản lý hạ tầng, luôn có model mới nhất. Nhưng **mất kiểm soát** về model version, data privacy, rate limit, pricing.
- **Self-hosting**: **kiểm soát tuyệt đối** – chọn phiên bản mô hình cụ thể, dữ liệu không rời khỏi tổ chức. Nhưng cần đội ngũ vận hành, GPU, tự xử lý scaling.
- **Managed Vector DB vs Self-hosted**: Pinecone serverless không cần quản lý nhưng giới hạn customization; Milvus self-hosted linh hoạt nhưng cần người vận hành.

Chiến lược cân bằng

Chiến lược Hybrid: managed service cho tác vụ thông thường, self-host cho các thành phần liên quan đến dữ liệu nhạy cảm hoặc yêu cầu đặc thù cao.

Trade-off 5: Tối ưu cục bộ – Tối ưu toàn cục

- **Cạm bẫy**: tối ưu từng thành phần riêng lẻ mà quên bức tranh tổng thể.

Ví dụ sai lầm

- Retrieval recall 99% $\rightarrow$ độ trễ pipeline gấp đôi $\rightarrow$ UX tệ hơn.
- Tiết kiệm 50% chi phí API (dùng mô hình rẻ) $\rightarrow$ tỉ lệ lỗi tăng $\rightarrow$ người dùng rời bỏ.
- GPU mạnh nhất để self-host $\rightarrow$ latency thấp nhất $\rightarrow$ nhưng GPU chỉ dùng 20% thời gian.

Chiến lược

- Đánh giá dựa trên chỉ số **end-to-end**: thời gian phản hồi tổng thể, tỉ lệ hài lòng, tổng chi phí.
- Thiết lập **SLO** ở mức hệ thống, để các quyết định tối ưu thành phần phục vụ SLO đó.

Khung ra quyết định kiến trúc 5 bước

1. **Xác định yêu cầu và ràng buộc**
   - Liệt kê yêu cầu chức năng, phi chức năng, ràng buộc. Phân loại "must-have" và "nice-to-have".
2. **Liệt kê các phương án**
   - Với mỗi quyết định, liệt kê ít nhất 2–3 phương án. Đừng chỉ xem xét phương án "phổ biến".
3. **Phân tích trade-off cho từng phương án**
   - Tác động lên: chất lượng, độ trễ, chi phí, độ phức tạp, khả năng mở rộng, bảo mật, khả năng bảo trì.
4. **Chọn phương án và ghi lại lý do (ADR)**
   - **Luôn ghi lại lý do** – Architecture Decision Record. Quan trọng cho onboarding và đánh giá lại sau này.
5. **Xác nhận và điều chỉnh**
   - Triển khai, đo lường thực tế, sẵn sàng điều chỉnh nếu thực tế khác giả định.

ADR – Architecture Decision Record

Tiêu đề	Mô tả ngắn gọn về quyết định (vd: "Sử dụng pgvector thay vì Pinecone vector search")
Trạng thái	Đề xuất / Đã chấp nhận / Đã triển khai / Đã thay thế
Ngày	Ngày ra quyết định
Bối cảnh	Mô tả vấn đề cần giải quyết, các ràng buộc và yêu cầu liên quan.
Các phương án	Liệt kê các phương án đã xem xét cùng ưu/nhược điểm.
Quyết định	Phương án được chọn và lý do chi tiết.
Hệ quả	Tác động: tích cực, tiêu cực, rủi ro, và việc cần làm tiếp theo.

ADR không chỉ giúp onboarding thành viên mới – còn là công cụ để **đánh giá lại quyết định khi hoàn cảnh thay đổi**. Nhiều quyết định "sai lầm" thực ra là quyết định đúng đắn tại thời điểm nó được đưa ra.

Thực hành: Bài toán Cố vấn học tập AI (1/2)

Yêu cầu: Trường đại học xây dựng hệ thống "Cố vấn học tập AI":

- Đề xuất khóa học dựa trên ngành học, sở thích, kết quả học tập, yêu cầu tốt nghiệp.
- Hỏi đáp về nội dung, yêu cầu và đánh giá của từng khóa học.
- Lập kế hoạch học tập cho từng học kỳ, đảm bảo điều kiện tiên quyết.

Ràng buộc:

- Dữ liệu khóa học và sinh viên **nhạy cảm** – phải tuân thủ bảo vệ dữ liệu cá nhân.
- Ngân sách hạn chế – **không có GPU cluster** riêng.
- Thời gian: **3 tháng** (trước kỳ đăng ký môn học).
- Đội ngũ: 2 backend, 1 frontend – **không có kỹ sư ML** chuyên trách.

Thực hành: Bài toán Cố vấn học tập AI (2/2)

Phân tích và quyết định:

1. **Must-have**: bảo mật dữ liệu; trả lời chính xác dựa trên dữ liệu thực; trích dẫn nguồn; phản hồi < 5 giây.
2. **Phương án**:
   - A (API-based): quá đơn giản – không xử lý được hàng trăm khóa học trong context window.
   - B (RAG + API): **phù hợp nhất** – RAG với managed vector DB, LLM API.
   - C (RAG + Self-host): vượt ngân sách và năng lực đội ngũ.
3. **Quyết định**: API-based monolithic + RAG (hybrid).
   - LLM: Claude Sonnet (câu phức tạp) + GPT-4o mini (câu đơn giản).
   - Vector DB: **pgvector** – tận dụng PostgreSQL hiện có của trường.
   - Backend: Python (FastAPI), Frontend: Next.js.
   - Triển khai: Docker Compose trên cloud VM.
4. **Kế hoạch**: Tuần 1–2 (prototype 10 khóa) $\rightarrow$ Tuần 3–4 (đánh giá độ trễ/chi phí) $\rightarrow$ Tuần 5–8 (mở rộng) $\rightarrow$ Tuần 9–12 (kiểm thử thực tế).

Tổng kết Section 4

Những điểm cần nhớ

1. **Bốn nguyên tắc thiết kế**: (1) Bắt đầu từ bài toán; (2) Thiết kế cho sự thay đổi; (3) Đơn giản hóa; (4) Ưu tiên UX.
2. **Năm trade-off cốt lõi**: Chất lượng – Độ trễ; Chi phí – Năng lực; Đơn giản – Linh hoạt; Kiểm soát – Tiện lợi; Tối ưu cục bộ – Toàn cục.
3. **Khung 5 bước + ADR**: ra quyết định có hệ thống, ghi lại lý do, sẵn sàng điều chỉnh.
4. **Kiến trúc là một quá trình**, không phải sự kiện một lần. Bắt đầu đơn giản, tiến hóa cùng sản phẩm.

Tự kiểm tra nhanh:

1. Phân biệt trade-off “Chất lượng – Độ trễ” và “Chi phí – Năng lực”.
2. ADR là gì? Tại sao nó quan trọng trong phát triển ứng dụng AI?

Tổng kết chương

Những điểm cần ghi nhớ

- **Kiến trúc AI tạo sinh khác biệt căn bản** với phần mềm truyền thống: tính không xác định của LLM, logic nằm trong trọng số mô hình, chi phí/độ trễ inference cao hơn nhiều bậc.
- **Kiến trúc phân tầng 5 tầng** (Trải nghiệm – Điều phối – AI/Mô hình – Dữ liệu – Hạ tầng) cung cấp khung tư duy hệ thống để thiết kế.
- **Ba mô hình kiến trúc**: API-based (MVP), Microservices (quy mô lớn), RAG-based (hỏi đáp tài liệu). Không loại trừ lẫn nhau, có thể kết hợp và tiến hóa.
- **Năm trade-off cốt lõi**: Chất lượng–Độ trễ, Chi phí–Năng lực, Đơn giản–Linh hoạt, Kiểm soát–Tiện lợi, Tối ưu cục bộ–Toàn cục. Mỗi quyết định kiến trúc là một sự đánh đổi có ý thức.
- **Khung ra quyết định 5 bước + ADR**: Kiến trúc là **quá trình liên tục** – đánh giá, quyết định, triển khai, đo lường, điều chỉnh